

# Mini-Projet TCSD

Charlène BROUTIER & Camille LAVERIE

Professeur Référent : Mohamed-Lamine Messai

---

Lien git du projet : [https://github.com/CamilleLV/LYON2\\_Cloud\\_Systems](https://github.com/CamilleLV/LYON2_Cloud_Systems)

Notebook Jupyter en fin de rapport

---

Université Lumière Lyon 2

Master 2 BI&A

# Table des matières :

|                                              |           |
|----------------------------------------------|-----------|
| <b>Utilisation de VMs (ANNULÉ ENSUITE) :</b> | <b>3</b>  |
| <b>Projet :</b>                              | <b>9</b>  |
| Création du Docker Compose :                 | 9         |
| <b>Partie Bonus :</b>                        | <b>15</b> |
| Impact sur la santé humaine :                | 16        |
| Impact sur la qualité des écosystèmes :      | 16        |
| Impact sur le changement climatique :        | 17        |
| Impact sur les ressources :                  | 17        |
| <b>Configuration de Kerberos :</b>           | <b>18</b> |
| <b>Conclusion</b>                            | <b>25</b> |
| <b>Jupyter Notebook complet :</b>            | <b>25</b> |

# Utilisation de VMs (ANNULÉ ENSUITE) :

Comme le titre de la section l'indique, nous sommes d'abord partis sur une solution de VMs dupliqués pour chaque DataNode et le NameNode, seulement, le PC n'arrivera pas à suivre donc dès qu'on a compris ces contraintes, nous avons changé de solution. Néanmoins, je trouvais cela intéressant de laisser cette partie projet puisque ça fait aussi partie de nos travaux de s'être trompé et d'avoir rectifié le tir ensuite. Si cela ne vous intéresse pas, vous pouvez directement aller à la section suivante.

## Installation d'Ubuntu 22.04.5 LTS :

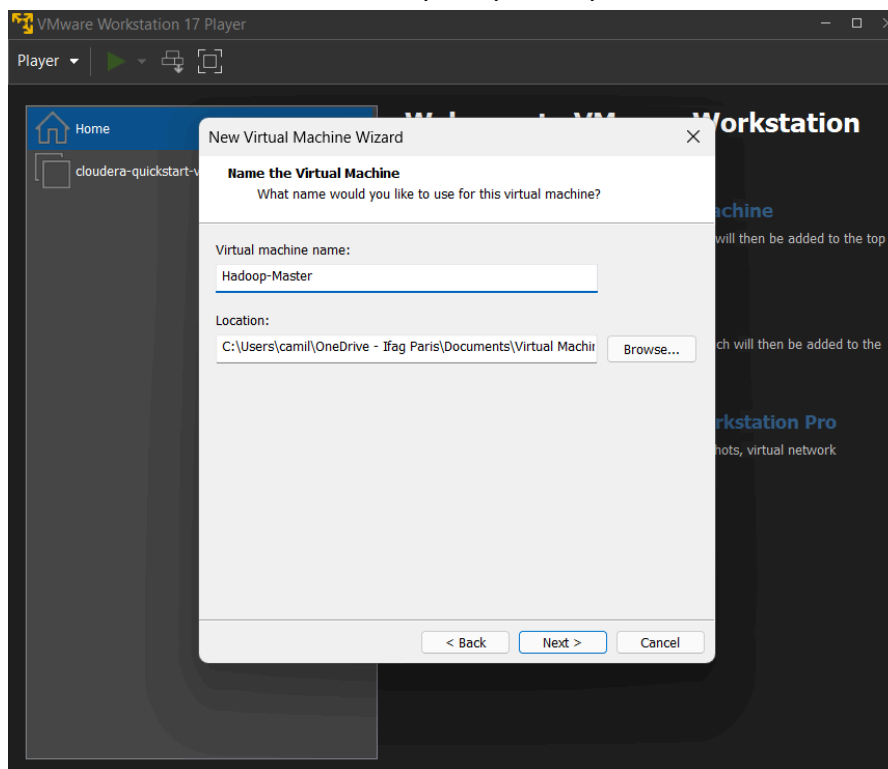
| Computers (file listing)                                                          |                                                              |                  |      |                                                                             |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------|------------------|------|-----------------------------------------------------------------------------|
|  | <a href="#">ubuntu-22.04.5-desktop-amd64.manifest</a>        | 2024-09-11 14:32 | 60K  | Desktop image for 64-bit PC (AMD64) computers (contents of live filesystem) |
|  | <a href="#">ubuntu-22.04.5-live-server-amd64.iso</a>         | 2024-09-11 18:46 | 2.0G | Server install image for 64-bit PC (AMD64) computers (standard download)    |
|  | <a href="#">ubuntu-22.04.5-live-server-amd64.iso.torrent</a> | 2024-09-12 18:16 | 160K | Server install image for 64-bit PC (AMD64) computers (BitTorrent download)  |
|  | <a href="#">ubuntu-22.04.5-live-server-amd64.iso.zsync</a>   | 2024-09-12 18:16 | 4.0M | Server install image for 64-bit PC (AMD64) computers (zsync metafile)       |
|  | <a href="#">ubuntu-22.04.5-live-server-amd64.list</a>        | 2024-09-11 18:46 | 8.5K | Server install image for 64-bit PC (AMD64) computers (file listing)         |

On va utiliser VMWare Workstation 17 "Player pour ce projet

En attendant l'installation du projet, création d'un repository git :

[https://github.com/CamilleLV/LYON2\\_Cloud\\_Systems](https://github.com/CamilleLV/LYON2_Cloud_Systems)

Création de la machine virtuelle principale depuis le fichier iso :



Hardware

| Device            | Summary                         |
|-------------------|---------------------------------|
| Memory            | 2 GB                            |
| Processors        | 2                               |
| New CD/DVD (SATA) | Using file C:\Users\camil\Co... |
| Network Adapter   | NAT                             |
| USB Controller    | Present                         |
| Sound Card        | Auto detect                     |
| Display           | Auto detect                     |

Memory

Specify the amount of memory allocated to this virtual machine. The memory size must be a multiple of 4 MB.

Memory for this virtual machine: 2048 MB

128 GB

64 GB

32 GB

16 GB

8 GB

4 GB

2 GB

1 GB

512 MB

256 MB

128 MB

64 MB

32 MB

16 MB

Maximum recommended memory  
(Memory swapping may occur beyond this size.)  
12.9 GB

Recommended memory  
4 GB

Guest OS recommended minimum  
2 GB

Configuration du stockage

[ Help ]

SOMMAIRE DU SYSTÈME DE FICHIERS

| POINT DE MONTAGE | TAILLE  | TYPE     | TYPE DE PÉRIPHÉRIQUE                  |
|------------------|---------|----------|---------------------------------------|
| [ /              | 10.000G | new ext4 | nouveau LVM logical volume ▶ ]        |
| [ /boot          | 1.771G  | new ext4 | nouveau partition de disque local ▶ ] |

DISQUES DISPONIBLES

| PÉRIPHÉRIQUE          | TYPE             | TAILLE      |
|-----------------------|------------------|-------------|
| [ ubuntu-vg (nouveau) | LVM volume group | 18.222G ▶ ] |
| espace libre          |                  | 8.222G ▶ ]  |

[ Create software RAID (md) ▶ ]

[ Create volume group (LVM) ▶ ]

PÉRIPHÉRIQUES UTILISÉS

| PÉRIPHÉRIQUE          | TYPE                                           | TAILLE      |
|-----------------------|------------------------------------------------|-------------|
| [ ubuntu-vg (nouveau) | LVM volume group                               | 18.222G ▶ ] |
| ubuntu-lv             | nouveau, to be formatted as ext4, mounted at / | 10.000G ▶ ] |

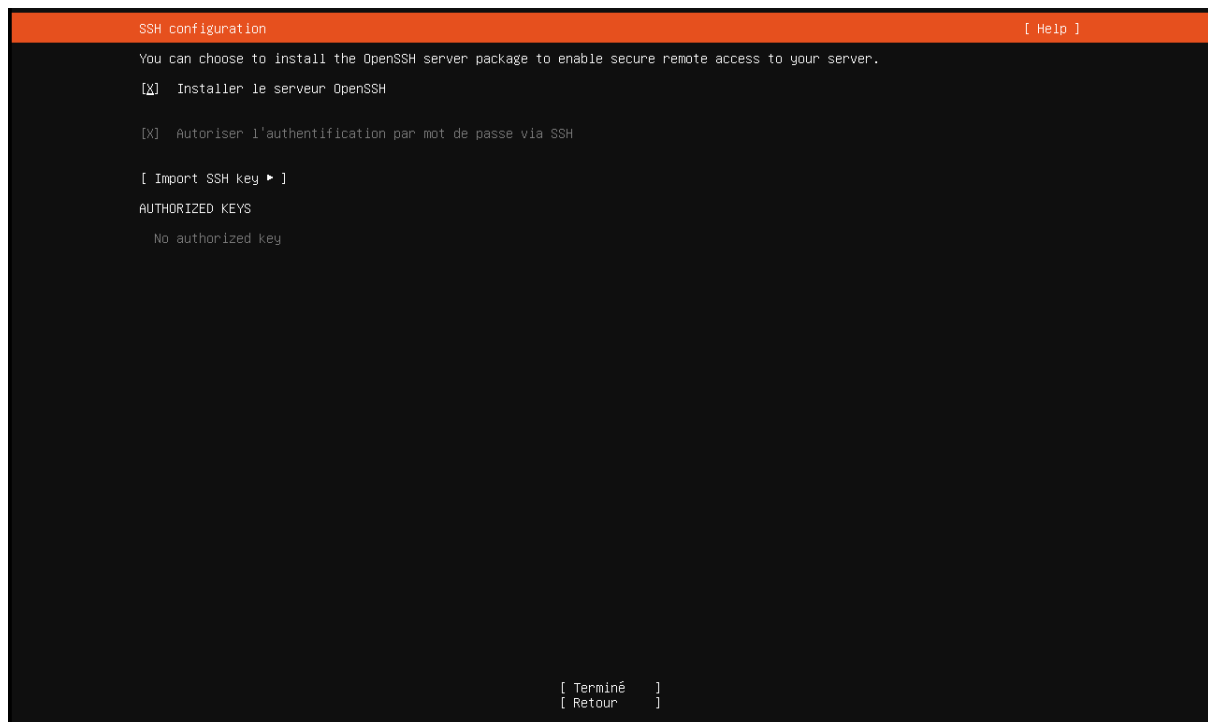
[ /dev/sda

| partition 1 | nouveau, BIOS grub spacer                          | 20.000G ▶ ] |
|-------------|----------------------------------------------------|-------------|
| partition 2 | nouveau, to be formatted as ext4, mounted at /boot | 1.000M ▶ ]  |
| partition 3 | nouveau, PV of LVM volume group ubuntu-vg          | 1.771G ▶ ]  |
|             |                                                    | 18.225G ▶ ] |

[ Terminé ]

[ Rétablir ]

[ Retour ]



J'ai bien validé "Installer le serveur OpenSSH"

Après finalisation de l'installation et redémarrage, nous avons accès à la VM :

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of jeu. 11 déc. 2025 15:18:38 UTC

System load:  0.05          Processes:           239
Usage of /:   49.5% of 9.75GB Users logged in:          0
Memory usage: 18%          IPv4 address for ens33: 192.168.40.131
Swap usage:   0%

La maintenance de sécurité étendue pour Applications n'est pas activée.

68 mises à jour peuvent être appliquées immédiatement.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
```

```
sudo apt update
```

## Installer les outils JAVA

```
sudo apt install net-tools nano default-jdk -y
```

## Vérifier qu'il y a Java

```
java -version
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
hadoop@hadoop-master:~$ java -version
openjdk version "11.0.29" 2025-10-21
OpenJDK Runtime Environment (build 11.0.29+7-post-Ubuntu-1ubuntu122.04)
OpenJDK 64-Bit Server VM (build 11.0.29+7-post-Ubuntu-1ubuntu122.04, mixed mode, sharing)
hadoop@hadoop-master:~$ _
```

## Générer la clé de sécurité SSH

```
ssh-keygen -t rsa -P ""
```

```
hadoop@hadoop-master:~$ ssh-keygen -t rsa -P ""
Generating public/private rsa key pair.
Enter file in which to save the key (/home/hadoop/.ssh/id_rsa):
Your identification has been saved in /home/hadoop/.ssh/id_rsa
Your public key has been saved in /home/hadoop/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:xBlXNoJUMLx0zt3einFvVRUbVdUQoyFwMnDtdvgqvVM hadoop@hadoop-master
The key's randomart image is:
+---[RSA 3072]-----+
|      .+X**.= +oO |
|      o+Oo+ + +. |
|      .+* o.. + |
|      .. + o ... |
|      S  o o .. |
|      . E o o |
|      . o + oo |
|      . + . ..o |
|      ..o  . |
+-----[SHA256]-----+
hadoop@hadoop-master:~$
```

## Autoriser la clé

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

## On va maintenant installer Hadoop

```
wget
```

```
https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

```
hadoop@hadoop-master:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$ wget https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
--2025-12-11 15:44:52-- https://dlcdn.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 730107476 (696M) [application/x-gzip]
Saving to: 'hadoop-3.3.6.tar.gz'

hadoop-3.3.6.tar.gz      16%[====>] 117,40M  8,70MB/s  eta 65s _
```

## Décompresser :

```
tar -xzf hadoop-3.3.6.tar.gz
```

```
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.2.4.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/hadoop-hdfs_0.21.0.xml
hadoop-3.3.6/share/hadoop/hdfs/jdiff/Apache_Hadoop_HDFS_3.1.3.xml
hadoop-3.3.6/share/hadoop/hdfs/hadoop-hdfs-client-3.3.6-tests.jar
hadoop-3.3.6/share/hadoop/hdfs/hadoop-hdfs-httpfs-3.3.6.jar
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$ _
```

Déplacer le dossier :

```
sudo mv hadoop-3.3.6 /usr/local/hadoop
```

Se donner les droits utilisateurs :

```
sudo chown -R hadoop:hadoop /usr/local/hadoop
```

```
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$ sudo mv hadoop-3.3.6 /usr/local/hadoop
hadoop@hadoop-master:~$ sudo chown -R hadoop:hadoop /usr/local/hadoop
hadoop@hadoop-master:~$ _
```

On va maintenant configurer le système :

Ouvrir le fichier de configuration

```
nano ~/.bashrc
```

On va y ajouter 4 lignes tout en bas du fichier :

```
export JAVA_HOME=/usr/lib/jvm/default-java
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

```
GNU nano 6.2 /home/hadoop/.bashrc *
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]} && echo terminal || echo error)" "${history}'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export JAVA_HOME=/usr/lib/jvm/default-java
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

Valider les changements :

```
export JAVA_HOME=/usr/lib/jvm/default-java
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

hadoop@hadoop-master:~$
hadoop@hadoop-master:~$
hadoop@hadoop-master:~$ source ~/.bashrc
hadoop@hadoop-master:~$ _
```

Vérification :

hadoop version

```
hadoop@hadoop-master:~$ source ~/.bashrc
hadoop@hadoop-master:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
hadoop@hadoop-master:~$
```

Maintenant que la VM est prête, on va simplement dupliquer le dossier de la VM pour faire les VMs esclaves, je les ai aussi déplacées dans le dossier C:/VM\_Hadoop pour ce projet car elles étaient situées sur Onedrive pour aucunes raisons.



# Projet :

Après réflexion sur la mémoire que cela demanderait de faire tourner 3 VM plutôt que des programmes Docker, et après échanges avec nos camarades, nous avons décidé de plutôt nous orienter sur la conteneurisation pour ce projet.

On reprend donc de zéro !

## Création du Docker Compose :

```
version: "3"

services:
  # --- NAME NODE (Le Chef) ---
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    hostname: namenode
    ports:
      - 9870:9870 # Interface Web
      - 9000:9000 # Port RPC pour Les datanodes et Spark
    volumes:
      - ./data/namenode:/hadoop/dfs/name
    environment:
      - CLUSTER_NAME=test
      - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
      - HDFS_CONF_dfs_replication=1 # Important: 1 seule copie car petit cluster
      - HDFS_CONF_dfs_permissions_enabled=false # Évite les erreurs de droits sur
Windows
```

```
# --- DATA NODE 1 (Ouvrier 1) ---
datanode1:
  image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
  container_name: datanode1
  depends_on:
    - namenode
  volumes:
    - ./data/datanode1:/hadoop/dfs/data
  environment:
    - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
    - SERVICE_PRECONDITION=namenode:9870

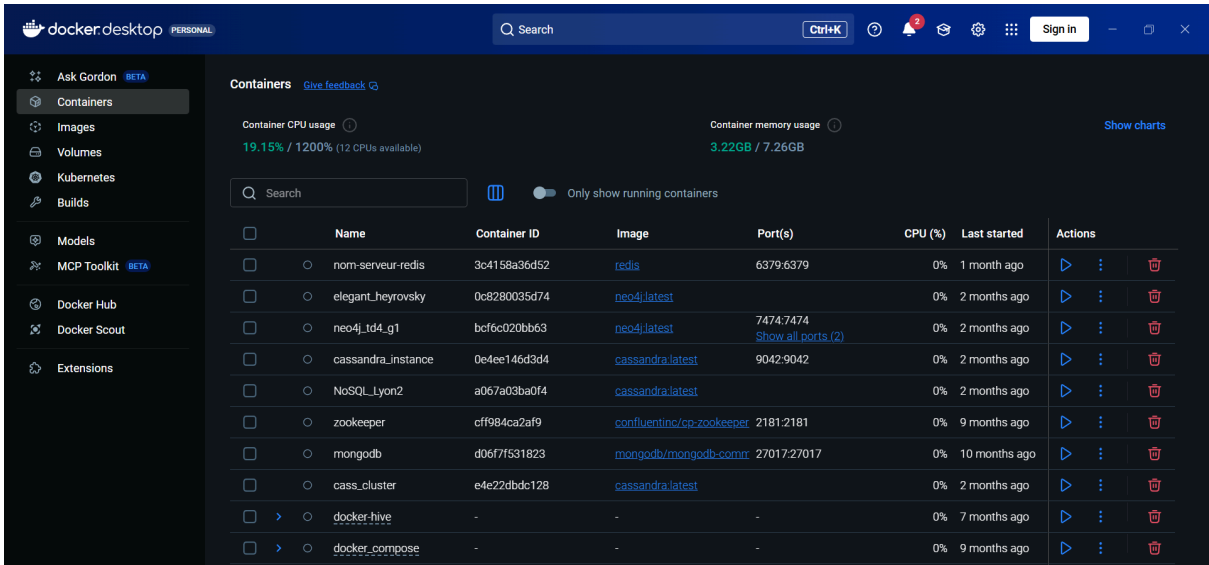
# --- DATA NODE 2 (Ouvrier 2) ---
datanode2:
  image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
```

```
container_name: datanode2
depends_on:
  - namenode
volumes:
  - ./data/datanode2:/hadoop/dfs/data
environment:
  - CORE_CONF_fs_defaultFS=hdfs://namenode:9000
  - SERVICE_PRECONDITION=namenode:9870

# --- SPARK CLIENT (Le Cerveau) ---
spark-client:
  image: jupyter/pyspark-notebook:latest
  container_name: spark-client
  ports:
    - "8888:8888"
    - "4040:4040"
  volumes:
    - ./input_data:/home/jovyan/work
  environment:
    - JUPYTER_TOKEN=bigdata
    # On dit à Spark où est le Namenode pour qu'il puisse lire les fichiers
    - SPARK_HADOOP_fs_defaultFS=hdfs://namenode:9000
  depends_on:
    - namenode

# --- KERBEROS ---
kerberos:
  image: gcavalcante8808/krb5-server
  container_name: kerberos
  hostname: kerberos.hadoop.docker
  environment:
    KRB5_REALM: "HADOOP.DOCKER"
    KRB5_KDC: "kerberos.hadoop.docker"
    KRB5_PASS: "password"
  ports:
    - "88:88"
    - "749:749"
  networks:
    default:
      aliases:
        - kerberos.hadoop.docker
```

Interface de Docker Desktop :



```
LYON2_PROJET_HADOOP
├── input_data
└── docker-compose.yml

docker-compose.yml > ...
1  version: "3"
2
3  Run All Services
4  services:
5    # --- MACHINE 1 : Le Chef (NameNode) ---
6    Run Service
7    namenode:
8      image: apache/hadoop:3
9      hostname: namenode
10     container_name: namenode
11     command: ["hdfs", "namenode"]
12     ports:
13       - 9870:9870 # Interface Web Hadoop
14     environment:
15       ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"
16     volumes:
17       - ./data/namenode:/tmp/hadoop-root/dfs/name
```

Puisque nous utiliserons Spark pour le traitement, nous avons créé le fichier docker-compose.yml qui contient du code qui créera ensuite 4 machines :

- Namenode : Le chef du stockage.
- Datanode1 & Datanode2 : Les ouvriers qui stockent les blocs de données.
- Jupyter-Spark : Le “poste de pilotage”.

Pour les données, nous souhaitons traiter des données concernant les films, on a trouvé un dataset Imdb qui contient leur TOP 1000 films :

## IMDB Movies Dataset

725 <> Code Download

Data Card Code (281) Discussion (5) Suggestions (0)

| About this file                                                                                           |                          |                                       |                                   |                                             |       |  |  | Summary    |    |
|-----------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------|-----------------------------------|---------------------------------------------|-------|--|--|------------|----|
| IMDB Dataset of Top 1000 Movies by IMDB Rating                                                            |                          |                                       |                                   |                                             |       |  |  | 1 file     |    |
|                                                                                                           |                          |                                       |                                   |                                             |       |  |  | 16 columns |    |
|                                                                                                           |                          |                                       |                                   |                                             |       |  |  | String     | 11 |
|                                                                                                           |                          |                                       |                                   |                                             |       |  |  | Integer    | 2  |
|                                                                                                           |                          |                                       |                                   |                                             |       |  |  | Decimal    | 2  |
|                                                                                                           |                          |                                       |                                   |                                             |       |  |  | Other      | 1  |
| Poster_Link                                                                                               | Series_Title             | Released_Year                         | Certificate                       | Runtime                                     | Genre |  |  |            |    |
| Poster Link of Movie                                                                                      | Name of the Movie        | Released Year of the Movie            | Certificate of the Movie          | Total Runtime of the Movie                  | Genre |  |  |            |    |
| 1000 unique values                                                                                        | 999 unique values        | 2014 3%<br>2004 3%<br>Other (937) 94% | U 23%<br>A 20%<br>Other (569) 57% | 130 min 2%<br>100 min 2%<br>Other (954) 95% | Drama |  |  |            |    |
| https://m.media-amazon.com/images/M/MV5BMDFkYTc0MGEtZmNhMC00ZDZlWFhNTk0ODM1ZmRlYWwvMWFmXkEyXkFqcGde...    | The Shawshank Redemption | 1994                                  | A                                 | 142 min                                     | Drama |  |  |            |    |
| https://m.media-amazon.com/images/M/MV5BM2MyNmJyYXNmUjYTAwNi00MTYxLWJmNWYtYzZlODY3ZTk3OTF1XkEyXkFqcGde... | The Godfather            | 1972                                  | A                                 | 175 min                                     | Crime |  |  |            |    |

Le fichier contient plusieurs colonnes inutiles, des données de plusieurs types donc on pourra les traiter en conséquence.

Lien Web :

<https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows?resource=download>

Maintenant que l'on a mis les données dans le dossier input\_data, on va pouvoir utiliser la commande suivante :

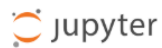
```
docker compose up -d
```

```
C:\LYON2_Projet_Hadoop>docker compose up -d
time="2025-12-16T15:08:03+01:00" level=warning msg="C:\\LYON2_Projet_Hadoop\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 4/56
- namenode [#####] 426.5MB / 942.2MB Pulling 262.6s
- datanode1 Pulling 262.6s
- spark-client [#####] 1.113GB / 1.808GB Pulling 262.6s
- datanode2 Pulling 262.6s
```

Fini !

```
C:\LYON2_Projet_Hadoop>docker compose up -d
time="2025-12-16T15:08:03+01:00" level=warning msg="C:\\LYON2_Projet_Hadoop\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 56/56
✓namenode Pulled 454.4s
✓datanode1 Pulled 454.4s
✓spark-client Pulled 450.6s
✓datanode2 Pulled 454.4s
```

Maintenant on va pouvoir lancer <http://localhost:8888> :



Password or token:

Log in

## Token authentication is enabled

If no password has been configured, you need to open the server with its login token in the URL, or paste it above. This requirement will be lifted if you [enable a password](#).

The command:

```
jupyter server list
```

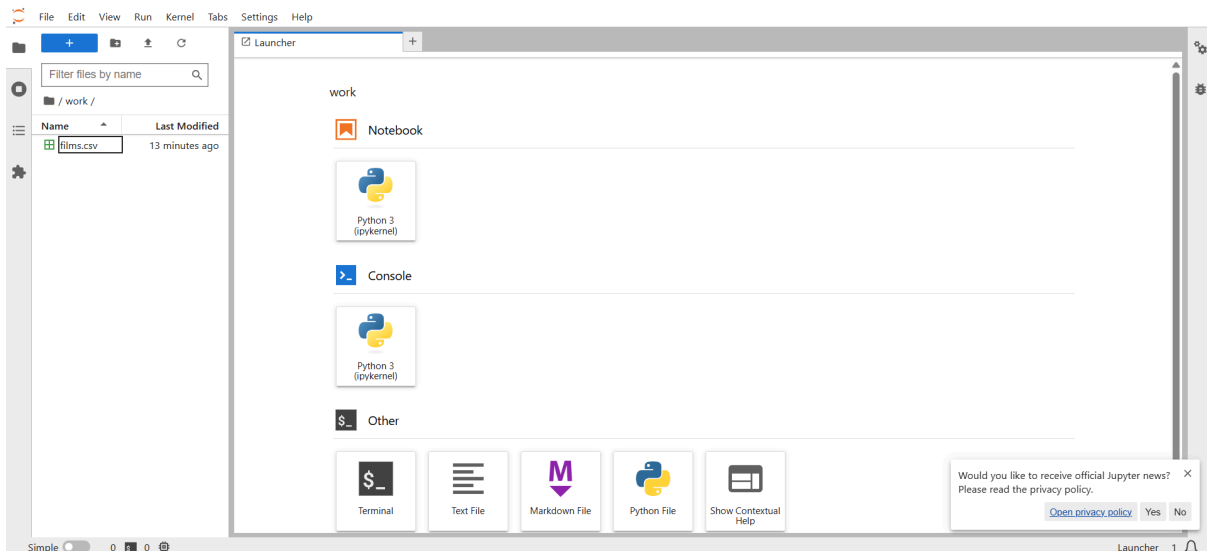
will show you the URLs of running servers with their tokens, which you can copy and paste into your browser. For example:

```
Currently running servers:  
http://localhost:8888/?token=c8de56fa... :: /Users/you/notebooks
```

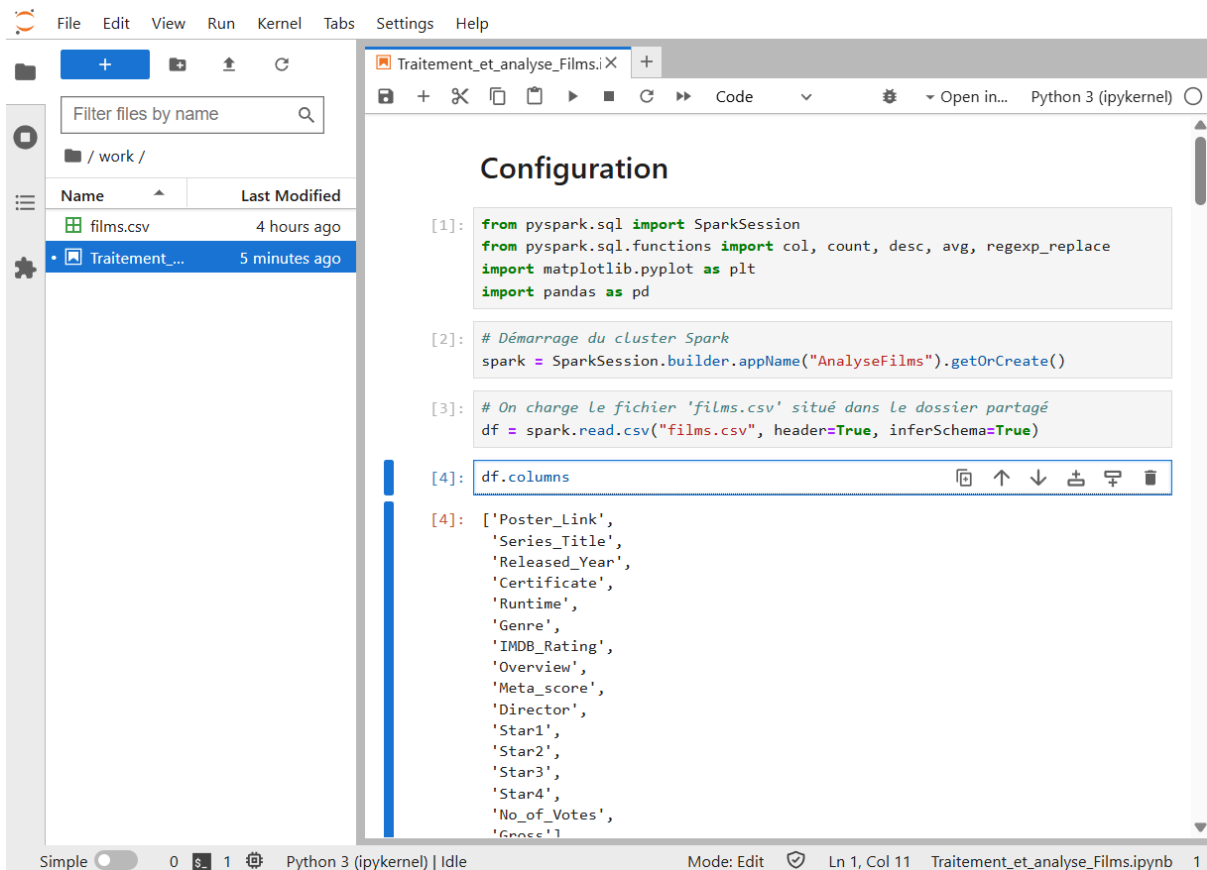
Le mot de passe est "bigdata" :

```
spark-client:  
image: jupyter/pyspark-notebook:latest  
container_name: spark-client  
ports:  
  - "8888:8888" # Accès au Notebook  
  - "4040:4040" # Monitoring Spark  
volumes:  
  - ./input_data:/home/jovyan/work # Lien entre Windows et le container  
environment:  
  - JUPYTER_TOKEN=bigdata # Mot de passe simple  
depends_on:  
  - namenode
```

Dans Work, on a directement le dataset films.csv que l'on a téléchargé précédemment :



Le notebook est complété, il est disponible dans le dossier input\_data :



Il y a un pdf du notebook complet à la racine du projet.

## Partie Bonus :

Dans cette partie bonus, nous allons traiter d'abord de la partie écologie : le calcul d'impact matériel de nos calculs :

Pour les constantes, nous sommes parti des caractéristiques de l'ordinateur de Camille (Victus 15FB) :

- Batterie Li-ion polymère 3 cellules de 52,5 Wh
- Type d'alimentation électrique avec adaptateur secteur 120 W
- Sources référence carbone : EcoInfo CNRS, GreenIT.fr

Voici les résultats :

```
--- RAPPORT D'IMPACT ÉCOLOGIQUE (Données Matérielles) ---  
Matériel : PC Portable avec alimentation 120W  
Batterie : Li-ion polymère 52.5 Wh  
Charge de travail : Cluster Hadoop/Spark (Estimation charge : 65%)  
Durée du traitement : 10.0 Heures  
-----  
Puissance moyenne estimée : 78.0 Watts  
Consommation Électrique : 0.7800 kWh  
Empreinte Carbone : 44.46 gCO2e  
-----  
Méthodologie : Calcul basé sur la puissance de l'adaptateur secteur pondérée  
par l'usage CPU/RAM intensif des conteneurs Docker.
```

Ces informations sont prises en compte suite à des calculs disponibles dans le Notebook, nous avons considéré un temps de travail d'une dizaine d'heures sur l'ordinateur, à écrire et tester le notebook, les résultats correspondent donc au temps passé avec ce matériel.

44 grammes de CO2 représentent peu de choses :

- Rejet moyen d'un pot d'échappement d'une voiture essence sur une distance de 400 mètres
- Rejet du corps humain moyen en une heure
- Quelques minutes d'utilisation d'un sèche-cheveux
- Envoi de 11 E-mails standards sans pièces jointes

L'impact écologique n'est pas le même dans nos recherches suivantes, en reprenant l'impactomètre indiqué dans l'énoncé, nous visitons un site permettant d'effectuer des comparaisons liés à des réunions (Skype, Zoom, etc), pour imaginer notre scénario d'utilisation du matériel pour ce projet avec une dizaine d'heures indicatives, nous avons mis l'équivalent de deux personnes en réunion de 10H (correspondant à Charlène et Camille pour ce projet) :



Voici les comparaisons que l'on a pu avoir :

## Impact sur la santé humaine :

L'impact sur la santé humaine est une catégorie de dommage environnemental qui s'exprime en DALY, Disability Adjusted Life Years (années de vie corrigées du facteur d'incapacité). Elle prend en compte les indicateurs environnementaux de la toxicité humaine, des effets respiratoires, des rayonnements ionisants et la diminution de la couche d'ozone.

Sur le plan de l'Impact sur la santé humaine :

Le Scénario A équivaut à l'impact de 2.9 km réalisés dans une voiture thermique.

## Impact sur la qualité des écosystèmes :

L'impact sur la qualité de l'écosystème est une catégorie de dommage environnemental qui s'exprime en PDF\*m<sup>2</sup>\*année. Le Potentially Disappeared Fraction of species (PDF) correspond à la fraction des espèces qui a une forte probabilité de ne pas se trouver dans une région. Cette catégorie prend en compte la diminution de la couche d'ozone, l'oxydation photochimique, l'écotoxicité aquatique et terrestre, l'acidification et l'eutrophisation des eaux et des terres, ainsi que l'occupation des terres.

Sur le plan de l'Impact sur la qualité des écosystèmes :

Le Scénario A équivaut à l'impact de 2.7 km réalisés dans une voiture thermique.



## Impact sur le changement climatique :

L'impact sur le changement climatique s'exprime en kilogramme équivalent CO<sub>2</sub> (KgeqCO<sub>2</sub>). Cette catégorie prend en compte les émissions des gaz à effet de serre.

Sur le plan de l'Impact sur le changement climatique :

Le Scénario A équivaut à l'impact de 1.4 km réalisés dans une voiture thermique.

## Impact sur les ressources :

L'impact sur les ressources est une catégorie de dommage environnemental qui s'exprime en MJ primaire. Cela regroupe à la fois l'utilisation d'énergies non renouvelables et l'extraction de minéraux.

Sur le plan de l'Impact sur les ressources :

Le Scénario A équivaut à l'impact de 3.6 km réalisés dans une voiture thermique.

# Configuration de Kerberos :

On va d'abord créer une copie de notre docker-compose qui est actuellement fonctionnel : docker-compose-backup.yml

Contrairement à une authentification simple, il permet une authentification forte et mutuelle entre un client et un serveur sur un réseau non sécurisé, en utilisant la cryptographie symétrique. Son objectif principal dans un cluster Hadoop est de garantir que chaque utilisateur et chaque service (Namenode, Datanode) est bien celui qu'il prétend être.

L'architecture repose sur trois entités :

- Le principal, l'identité unique de l'utilisateur ou du service
- L'Authentication Service (AS) qui vérifie l'identité initiale
- Le Ticket Granting Service (TGS) qui délivre les tickets d'accès au service

Ces deux dernières entités sont donc regroupées dans le KDC (Key Distribution Center), l'autorité centrale de sécurité qui sera installée sur notre cluster si tout se passe bien !

On modifie donc le docker-compose.yml pour y ajouter le kerberos :

```
51  # --- KERBEROS ---
52  >Run Service
53  kerberos:
54    image: gcavalcante8808/krb5-server
55    container_name: kerberos
56    hostname: kerberos.hadoop.docker
57    environment:
58      KRB5_REALM: "HADOOP.DOCKER"
59      KRB5_KDC: "kerberos.hadoop.docker"
60      KRB5_PASS: "password" # Mot de passe admin
61    ports:
62      - "88:88"          # Port Ticket
63      - "749:749"        # Port Admin
64    networks:
65      default:
66        aliases:
67          - kerberos.hadoop.docker
```

```

C:\LYON2_Projet_Hadoop>
C:\LYON2_Projet_Hadoop>docker compose up -d
time="2026-01-05T16:17:53+01:00" level=warning msg="C:\\LYON2_Projet_Hadoop\\docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 6/6
  ✓ kerberos Pulled                                14.5s
  ✓ 237d90a19ded Pull complete                      1.2s
  ✓ 4bc4c642713e Pull complete                      9.6s
  ✓ 110e12dbe0a3 Pull complete                      1.4s
  ✓ 661ff4d9561e Pull complete                      2.4s
  ✓ 8e1d69373fc5 Download complete                  0.0s
[+] Running 5/5
  ✓ Container spark-client Running                  0.0s
  ✓ Container kerberos Started                     0.9s
  ✓ Container namenode Started                     0.4s
  ✓ Container datanode1 Started                    0.4s
  ✓ Container datanode2 Started                     0.4s

```

Il faut maintenant devenir administrateur (il y a pas de bash pour ça qu'on utilise que /sh) :

Accéder au conteneur :

```
docker exec -it kerberos /bin/sh
```

On lance l'administration Kerberos :

```
kadmin -p admin/admin
```

(Le mot de passe c'est password)

Lorsque le prompt est en kadmin, taper :

```
addprinc camil
```

(On a mis le mot de passe camil)

```

C:\LYON2_Projet_Hadoop>docker exec -it kerberos /bin/sh
/ #
/ #
/ # kadmin -p admin/admin
Authenticating as principal admin/admin with password.
Password for admin/admin@HADOOP.DOCKER:
kadmin: addprinc camil
No policy specified for camil@HADOOP.DOCKER; defaulting to no policy
Enter password for principal "camil@HADOOP.DOCKER":
Re-enter password for principal "camil@HADOOP.DOCKER":
Principal "camil@HADOOP.DOCKER" created.
kadmin: listprincs
K/M@HADOOP.DOCKER
admin/admin@HADOOP.DOCKER
camil@HADOOP.DOCKER
kadmin/admin@HADOOP.DOCKER
kadmin/changepw@HADOOP.DOCKER
krbtgt/HADOOP.DOCKER@HADOOP.DOCKER
kadmin: |

```

Ensuite, on va pouvoir lancer la cellule Python de test dans Jupyter !

## PARTIE 3 (BONUS) : Test du Kerberos

Installation du client Kerberos (Commande Linux via Python) On installe les outils pour parler au serveur de sécurité

```
print("Installation du client Kerberos en cours...")
os.system("sudo apt-get update > /dev/null")
os.system("sudo DEBIAN_FRONTEND=noninteractive apt-get install -y krb5-user > /dev/null")
```

Installation du client Kerberos en cours...

256

Il y a un code d'erreur 256 : Kerberos ne s'est pas installé via Python donc la commande kinit "n'existe pas". On va installer le client manuellement :

```
docker exec -u 0 -it spark-client /bin/bash
```

```
Microsoft Windows [version 10.0.26200.7462]
(c) Microsoft Corporation. Tous droits réservés.

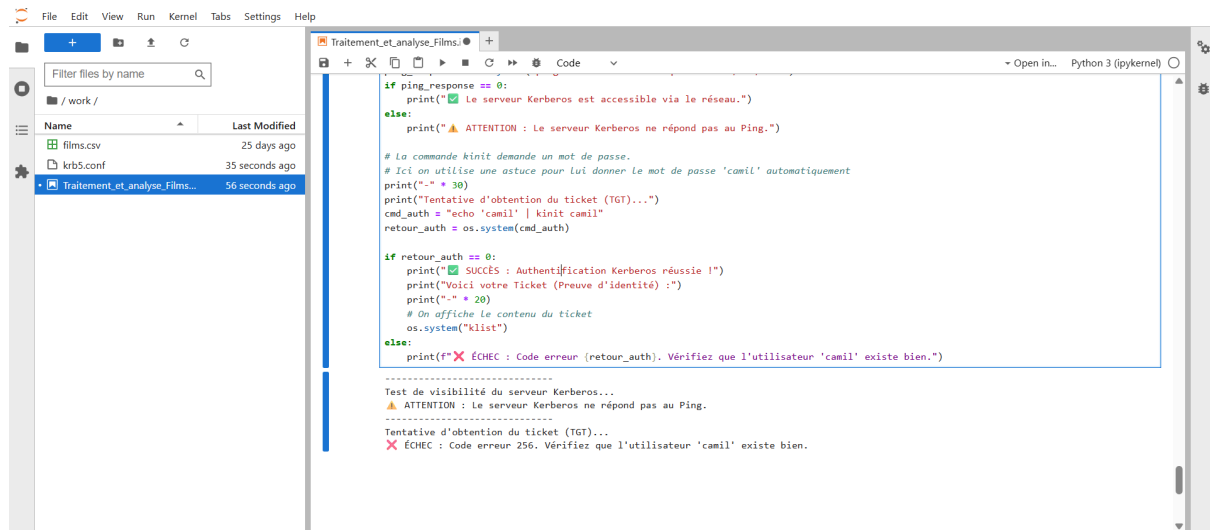
C:\Users\camil>docker exec -u 0 -it spark-client /bin/bash
(base) root@436bdc6e3ef3:~#
(base) root@436bdc6e3ef3:~#
(base) root@436bdc6e3ef3:~# |
```

puis :

```
apt-get update && apt-get install -y krb5-user
```

```
(base) root@436bdc6e3ef3:~# apt-get update && apt-get install -y krb5-user
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:5 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,637 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,600 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,968 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,289 kB]
Fetched 10.9 MB in 1s (11.6 MB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
krb5-user is already the newest version (1.19.2-2ubuntu0.7).
0 upgraded, 0 newly installed, 0 to remove and 143 not upgraded.
(base) root@436bdc6e3ef3:~# |
```

Après modification du code, nous sommes toujours bloqué :



Le problème viendrait peut-être du `krb5_config`, on changera donc “`kerberos.hadoop.docker`” par juste “`kerberos`” :

```

krb5_config = """
[libdefaults]
    default_realm = HADOOP.DOCKER
    dns_lookup_realm = false
    dns_lookup_kdc = false

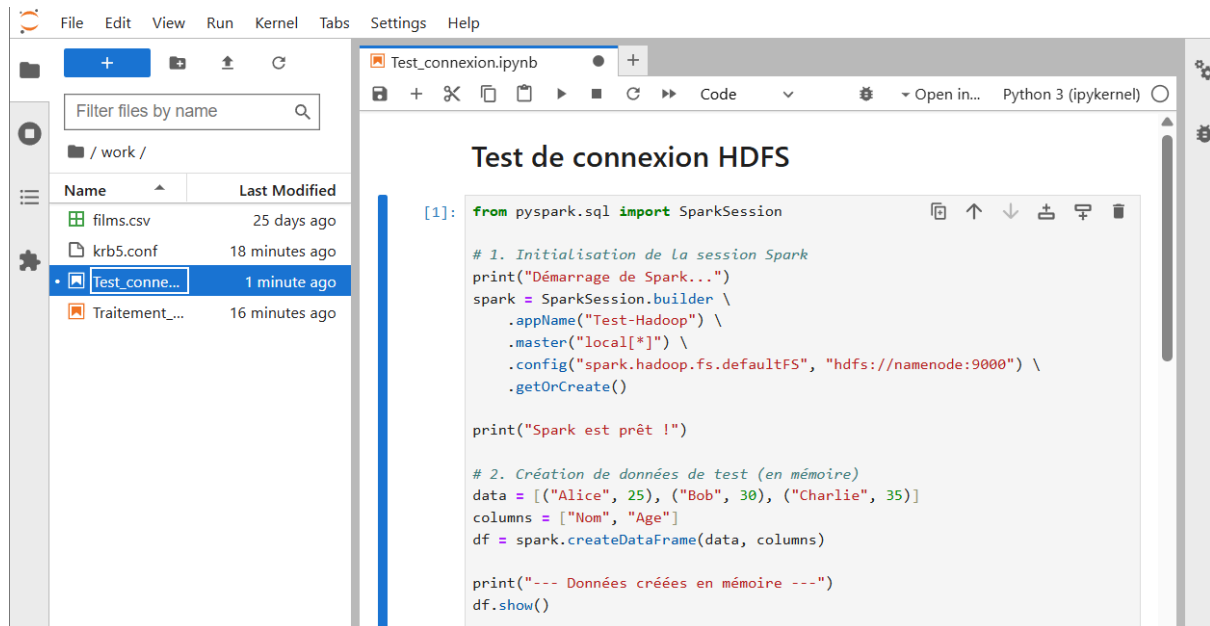
[realms]
    HADOOP.DOCKER = {
        kdc = kerberos
        admin_server = kerberos
    }

[domain_realm]
    .hadoop.docker = HADOOP.DOCKER
    hadoop.docker = HADOOP.DOCKER
"""

```

Puisque le projet bloque, j’ai refais complètement le `docker-compose.yml`, vous pourrez donc voir la version sans Kerberos (`docker-compose-backup.yml`) et la version avec kerberos avant la refonte du fichier (`docker-compose-V1.yml`).

Avec la nouvelle version de mon projet, j'ai généré un test de connexion HDFS :



```
[1]: from pyspark.sql import SparkSession

# 1. Initialisation de la session Spark
print("Démarrage de Spark...")
spark = SparkSession.builder \
    .appName("Test-Hadoop") \
    .master("local[*]") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000") \
    .getOrCreate()

print("Spark est prêt !")

# 2. Création de données de test (en mémoire)
data = [("Alice", 25), ("Bob", 30), ("Charlie", 35)]
columns = ["Nom", "Age"]
df = spark.createDataFrame(data, columns)

print("--- Données créées en mémoire ---")
df.show()
```

Résultat :

```
Démarrage de Spark...
Spark est prêt !
--- Données créées en mémoire ---
+-----+-----+
|    Nom|Age|
+-----+-----+
|  Alice| 25|
|   Bob| 30|
|Charlie| 35|
+-----+-----+

Écriture des données vers : hdfs://namenode:9000/user/test_write ...
Écriture terminée !
Lecture des données depuis Hadoop...
+-----+-----+
|   _c0|_c1|
+-----+-----+
|Charlie| 35|
|  Alice| 25|
|   Bob| 30|
+-----+-----+

Succès ! La connexion Spark <-> Hadoop fonctionne.
```

(C'est indiqué \_c0 et \_c1 car spark a donné des noms par défaut aux colonnes)

On re-installe kinit et on crée de nouveau l'utilisateur 'camil' :

```
C:\LYON2_Projet_Hadoop>docker exec -it kerberos kadmin.local -q "addprinc -pw camil camil"
Authenticating as principal root/admin@HADOOP.DOCKER with password.
No policy specified for camil@HADOOP.DOCKER; defaulting to no policy
Principal "camil@HADOOP.DOCKER" created.

C:\LYON2_Projet_Hadoop>
```

On relance tout et tout semble fonctionner :

|                          |   |              |                     |             |                                                                   |         |      |                          |   |
|--------------------------|---|--------------|---------------------|-------------|-------------------------------------------------------------------|---------|------|--------------------------|---|
| <input type="checkbox"/> | ▼ | ●            | lyon2_projet_hado - | -           | -                                                                 | 204.74% | 18 m | <input type="checkbox"/> | ⋮ |
| <input type="checkbox"/> | ● | spark-client | 306666b3a558        | jupyter/pys | <a href="#">4040:4040 ↗</a><br><a href="#">Show all ports (2)</a> | 204.13% | 18 m | <input type="checkbox"/> | ⋮ |
| <input type="checkbox"/> | ● | datanode1    | 8db484b3eff7        | bde2020/h   |                                                                   | 0.23%   | 18 m | <input type="checkbox"/> | ⋮ |
| <input type="checkbox"/> | ● | datanode2    | 302248f52c3c        | bde2020/h   |                                                                   | 0.22%   | 18 m | <input type="checkbox"/> | ⋮ |
| <input type="checkbox"/> | ● | kerberos     | e1141bd39904        | gcavalcant  | <a href="#">749:749 ↗</a><br><a href="#">Show all ports (2)</a>   | 0.01%   | 18 m | <input type="checkbox"/> | ⋮ |
| <input type="checkbox"/> | ● | namenode     | b503b7b7e7ad        | bde2020/h   | <a href="#">9000:9000 ↗</a><br><a href="#">Show all ports (2)</a> | 0.15%   | 18 m | <input type="checkbox"/> | ⋮ |

Petite erreur d'écriture système :

```

16 """
18 # On écrit cette configuration dans le fichier système
--> 19 with open("/etc/krb5.conf", "w") as f:
20     f.write(krb5_config)
21 print("Configuration Kerberos (/etc/krb5.conf) mise à jour.")

File /opt/conda/lib/python3.11/site-packages/IPython/core/interactiveshell.
py:286, in _modified_open(file, *args, **kwargs)
279 if file in {0, 1, 2}:
280     raise ValueError(
281         f"IPython won't let you open fd={file} by default "
282         "as it is likely to crash IPython. If you know what you are
doing, "
283         "you can use builtins' open."
284     )
--> 286 return io_open(file, *args, **kwargs)

PermissionError: [Errno 13] Permission denied: '/etc/krb5.conf'

```

On autorise la lecture/écriture :

```
docker exec -u 0 -it spark-client bash -c "touch /etc/krb5.conf && chmod 666 /etc/krb5.conf"
```

Résultat final :

```
Configuration Kerberos (/etc/krb5.conf) mise à jour.
-----
Tentative d'obtention du ticket (TGT)...
✅ SUCCÈS : Authentification Kerberos réussie !
Voici votre Ticket (Preuve d'identité) :
-----
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: camil@HADOOP.DOCKER

Valid starting      Expires            Service principal
01/11/2026 11:56:53 01/11/2026 23:56:53 krbtgt/HADOOP.DOCKER@HADOOP.DOCKER
renew until 01/12/2026 11:56:53
```

Ici, dans les faits, il y a un Kerberos, mais avec ou sans, on peut toujours utiliser le programme, il faudrait créer des fichiers de clés (Keytabs) complexes pour chaque conteneur, ce serait sûrement trop lourd pour le cluster Docker actuel. Je vais par contre modifier la configuration spark pour qu'il exige un ticket kerberos valide :

```
# Démarrage du cluster Spark en mode STRICT Kerberos
# On force le client à utiliser Kerberos
spark = SparkSession.builder \
    .appName("AnalyseFilms_Kerberos") \
    .master("local[*]") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000") \
    .config("spark.hadoop.hadoop.security.authentication", "kerberos") \
    .config("spark.hadoop.ipc.client.fallback-to-simple-auth-allowed", "false") \
    .getOrCreate()

print("Cluster Spark démarré en mode SÉCURISÉ !")
```

Cluster Spark démarré en mode SÉCURISÉ !

Comme indiqué sur le Notebook, le programme tournera de lui-même quand même, dû au Fallback to Simple Auth, "si le serveur ne comprend pas mon badge de sécurité, je rentre quand même sans badge".

Afin de sécuriser le cluster, nous avons déployé un serveur KDC (Key Distribution Center). Nous avons configuré le client Spark pour s'authentifier auprès de ce serveur et



obtenir un TGT (Ticket Granting Ticket), prouvant ainsi l'identité de l'utilisateur avant tout traitement.

## Conclusion

Ce projet nous a permis de mettre en œuvre une chaîne de traitement Big Data complète, allant du déploiement de l'infrastructure jusqu'à l'analyse de données, en passant par la sécurisation des accès. Dans un premier temps, nous avons réussi à stabiliser un cluster Hadoop distribué via Docker, surmontant les contraintes techniques liées à l'environnement Windows. Cela nous a permis ensuite d'utiliser Apache Spark, pour analyser après traitement les données sur les films. La dimension sécuritaire a ensuite constitué un aspect central du projet avec l'intégration du protocole Kerberos. Nous avons configuré un serveur d'authentification complet et géré l'attribution des tickets pour contrôler strictement les accès au cluster. En conclusion, nous avons réussi à accomplir ce qui était demandé, en simulant les contraintes réelles d'un environnement de production d'une entreprise.

Jupyter Notebook complet :

# Traitement\_et\_analyse\_Films

January 11, 2026

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, desc, avg, regexp_replace
import matplotlib.pyplot as plt
import pandas as pd

#(pour Kerberos plus bas)
import os
import subprocess
```

## 0.1 PARTIE 3 (BONUS) : Test du Kerberos

Installation du client Kerberos (Commande Linux via Python) On installe les outils pour parler au serveur de sécurité

Non fonctionnel (donc installation manuelle, CF. rapport)

Configuration du fichier krb5.conf (L'annuaire) On dit au client où se trouve le serveur Kerberos (notre conteneur 'kerberos')

```
[2]: krb5_config = """
[libdefaults]
    default_realm = HADOOP.DOCKER
    dns_lookup_realm = false
    dns_lookup_kdc = false

[realms]
    HADOOP.DOCKER = {
        kdc = kerberos:88
        admin_server = kerberos:749
    }

[domain_realm]
    .hadoop.docker = HADOOP.DOCKER
    hadoop.docker = HADOOP.DOCKER
"""

# On écrit cette configuration dans le fichier système
with open("/etc/krb5.conf", "w") as f:
    f.write(krb5_config)
```

```

print("Configuration Kerberos (/etc/krb5.conf) mise à jour.")

#Authentification (Le test final)
#On essaie d'obtenir un ticket pour l'utilisateur 'camil'

# La commande kinit demande un mot de passe.
# Ici on utilise une astuce pour lui donner le mot de passe 'camil'
↳ automatiquement
print("-" * 30)
print("Tentative d'obtention du ticket (TGT)...")
cmd_auth = "echo 'camil' | kinit camil"

retour_auth = os.system(cmd_auth)

if retour_auth == 0:
    print(" SUCCÈS : Authentification Kerberos réussie !")
    print("Voici votre Ticket (Preuve d'identité) :")
    print("-" * 20)
    # On affiche le contenu du ticket
    resultat = subprocess.getoutput("klist")
    print(resultat)
else:
    print(f" ÉCHEC : Code erreur {retour_auth}. Vérifiez que l'utilisateur_
↳ 'camil' existe bien.")

```

Configuration Kerberos (/etc/krb5.conf) mise à jour.

```

-----
Tentative d'obtention du ticket (TGT)...
  SUCCÈS : Authentification Kerberos réussie !
Voici votre Ticket (Preuve d'identité) :
-----
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: camil@HADOOP.DOCKER

Valid starting    Expires    Service principal
01/11/2026 12:33:21  01/12/2026 00:33:21  krbtgt/HADOOP.DOCKER@HADOOP.DOCKER
        renew until 01/12/2026 12:33:21

```

## 0.2 Configuration

### 0.2.1 Test avec et sans ticket :

```

[3]: # !kdestroy
    !klist
    # Doit afficher : "No credentials cache found"

```

```

Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: camil@HADOOP.DOCKER

```

| Valid starting                  | Expires             | Service principal                  |
|---------------------------------|---------------------|------------------------------------|
| 01/11/2026 12:33:21             | 01/12/2026 00:33:21 | krbtgt/HADOOP.DOCKER@HADOOP.DOCKER |
| renew until 01/12/2026 12:33:21 |                     |                                    |

Cette simulation n'empêche toujours pas le Fallback to Simple Auth

```
[4]: # !echo 'camil' | kinit camil
```

```
[5]: # Démarrage du cluster Spark (ancienne version)
# spark = SparkSession.builder.appName("AnalyseFilms").getOrCreate()

# Démarrage du cluster Spark (nouvelle version)
# spark = SparkSession.builder \
#     .appName("AnalyseFilms_Kerberos") \
#     .master("local[*]") \
#     .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000") \
#     .getOrCreate()

# print("Cluster Spark démarré !")

# Démarrage du cluster Spark en mode STRICT Kerberos
# On force le client à utiliser Kerberos
spark = SparkSession.builder \
    .appName("AnalyseFilms_Kerberos") \
    .master("local[*]") \
    .config("spark.hadoop.fs.defaultFS", "hdfs://namenode:9000") \
    .config("spark.hadoop.hadoop.security.authentication", "kerberos") \
    .config("spark.hadoop.ipc.client.fallback-to-simple-auth-allowed", "false") \
    ↪\
    .getOrCreate()

print("Cluster Spark démarré en mode SÉCURISÉ !")
```

Cluster Spark démarré en mode SÉCURISÉ !

## 0.2.2 Suite de la configuration

```
[6]: # On charge le fichier 'films.csv' situé dans le dossier partagé
# df = spark.read.csv("films.csv", header=True, inferSchema=True)

# Lecture du fichier csv
# Note : On utilise file:/// pour lire le fichier local monté dans le volume
csv_path = "file:///home/jovyan/work/films.csv"

try:
    df = spark.read.csv(csv_path, header=True, inferSchema=True)
    print(f"Fichier chargé. {df.count()} lignes.")
```

```
df.show(5)
except Exception as e:
    print("Erreur de chargement :", e)
```

Fichier chargé. 1000 lignes.

```
+-----+-----+-----+-----+-----+-----+
|          Poster_Link|          Series_Title|Released_Year|Certificate|Runtime|
Genre|IMDB_Rating|          Overview|Meta_score|          Director|
Star1|          Star2|          Star3|          Star4|No_of_Votes|          Gross|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|https://m.media-a...|The Shawshank Red...|          1994|          A|142 min|
Drama|          9.3|Two imprisoned me...|          80|          Frank Darabont|          Tim
Robbins|Morgan Freeman|          Bob Gunton|William Sadler|          2343110|          28,341,469|
|https://m.media-a...|          The Godfather|          1972|          A|175 min|
Crime, Drama|          9.2|An organized crim...|          100|Francis Ford Coppola|
Marlon Brando|          Al Pacino|          James Caan|          Diane Keaton|
1620367|134,966,411|
|https://m.media-a...|          The Dark Knight|          2008|          UA|152
min|Action, Crime, Drama|          9.0|When the menace k...|          84|
Christopher Nolan|Christian Bale|          Heath Ledger|Aaron Eckhart|          Michael Caine|
2303232|534,858,444|
|https://m.media-a...|The Godfather: Pa...|          1974|          A|202 min|
Crime, Drama|          9.0|The early life an...|          90|Francis Ford Coppola|
Al Pacino|Robert De Niro|Robert Duvall|          Diane Keaton|          1129952|          57,300,000|
|https://m.media-a...|          12 Angry Men|          1957|          U|          96 min|
Crime, Drama|          9.0|A jury holdout at...|          96|          Sidney Lumet|
Henry Fonda|          Lee J. Cobb|Martin Balsam|          John Fiedler|          689845|          4,360,000|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+
only showing top 5 rows
```

```
[7]: df.columns
```

```
[7]: ['Poster_Link',
      'Series_Title',
      'Released_Year',
      'Certificate',
      'Runtime',
```

```
'Genre',
'IMDB_Rating',
'Overview',
'Meta_score',
'Director',
'Star1',
'Star2',
'Star3',
'Star4',
'No_of_Votes',
'Gross']
```

```
[8]: # Supprimer les colonnes inutiles
columns_to_drop = ["Poster_Link", "Overview", "Certificate", "Meta_score"]
df_clean = df.drop(*columns_to_drop)
```

```
[9]: # Nettoyer la colonne recette (Gross) détectée comme texte à cause des virgules
↳:
df_clean = df_clean.withColumn("Gross", regexp_replace(col("Gross"), ",", "")
↳cast("double"))
```

```
[10]: # Supprimer les lignes vides
df = df.dropna()
```

```
[11]: print("Aperçu des données :")
df.select("Series_Title", "Released_Year", "IMDB_Rating", "Genre", "Gross").
↳show(5)
```

Aperçu des données :

```
+-----+-----+-----+-----+
+
|      Series_Title|Released_Year|IMDB_Rating|      Genre|
Gross|
+-----+-----+-----+-----+
+
|The Shawshank Red...|      1994|      9.3|      Drama|
28,341,469|
|      The Godfather|      1972|      9.2|      Crime,
Drama|134,966,411|
|      The Dark Knight|      2008|      9.0|Action, Crime,
Drama|534,858,444|
|The Godfather: Pa...|      1974|      9.0|      Crime, Drama|
57,300,000|
|      12 Angry Men|      1957|      9.0|      Crime, Drama|
4,360,000|
+-----+-----+-----+-----+
+
```

only showing top 5 rows

### 0.3 Analyse 1 : Top 10 des réalisateurs et nombre de films dans le top 1000

```
[12]: print("--- Top 10 des Réalisateurs ---")

top_directors = df.groupBy("Director").count().orderBy(desc("count")).limit(10)

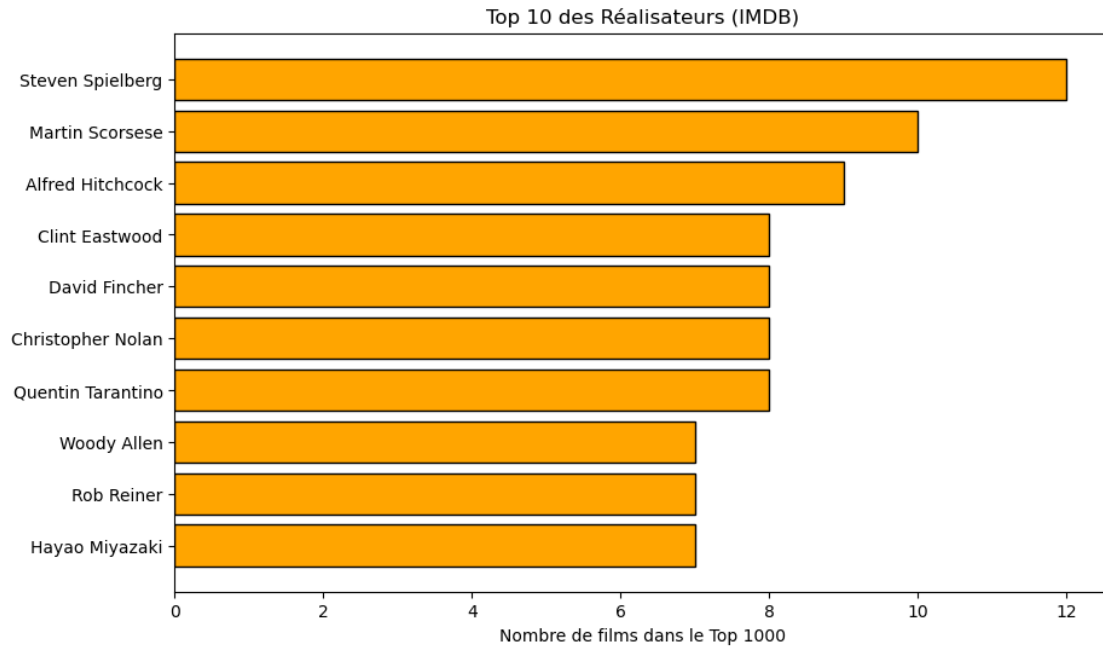
top_directors.show()
```

```
--- Top 10 des Réalisateurs ---
+-----+-----+
|      Director|count|
+-----+-----+
| Steven Spielberg| 12|
| Martin Scorsese| 10|
| Alfred Hitchcock| 9|
| Clint Eastwood| 8|
| David Fincher| 8|
| Christopher Nolan| 8|
| Quentin Tarantino| 8|
| Woody Allen| 7|
| Rob Reiner| 7|
| Hayao Miyazaki| 7|
+-----+-----+
```

#### Visualisation 1

```
[13]: pdf_directors = top_directors.toPandas()

[14]: plt.figure(figsize=(10, 6))
plt.barh(pdf_directors['Director'], pdf_directors['count'], color='orange',
        edgecolor='black')
plt.xlabel('Nombre de films dans le Top 1000')
plt.title('Top 10 des Réalisateurs (IMDB)')
plt.gca().invert_yaxis() # Pour avoir le premier en haut
plt.show()
```



#### 0.4 Analyse 2 : Top 10 des notes moyennes par genre

```
[15]: print("--- Note moyenne par Genre (Top 10) ---")

# Même si un film peut avoir plusieurs genre, j'ai décidé de simplifier la
# requête initiale
top_genres = df.groupby("Genre").agg(avg("IMDB_Rating").alias("Note_Moyenne")).
# orderBy(desc("Note_Moyenne")).limit(10)

top_genres.show()
```

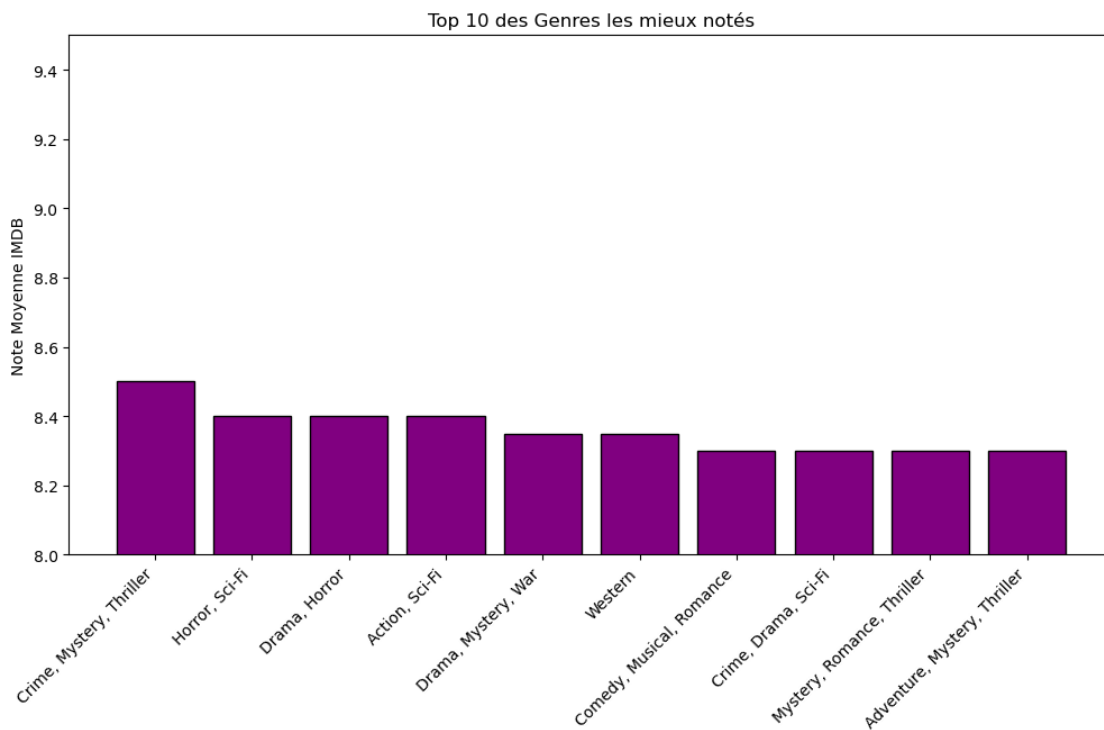
```
--- Note moyenne par Genre (Top 10) ---
+-----+-----+
|          Genre|    Note_Moyenne|
+-----+-----+
|Crime, Mystery, T...|      8.5|
|    Horror, Sci-Fi|      8.4|
|    Drama, Horror|      8.4|
|    Action, Sci-Fi|      8.4|
|Drama, Mystery, War|8.350000000000001|
|    Western|      8.35|
|Comedy, Musical, ...|      8.3|
|Crime, Drama, Sci-Fi|      8.3|
|Mystery, Romance,...|      8.3|
|Adventure, Myster...|      8.3|
+-----+-----+
```



## Visualisation 2

```
[16]: pdf_genres = top_genres.toPandas()

plt.figure(figsize=(12, 6))
plt.bar(pdf_genres['Genre'], pdf_genres['Note_Moyenne'], color='purple',
        edgecolor='black')
plt.xticks(rotation=45, ha='right') # On penche les textes pour lire
plt.ylim(8, 9.5) # On zoom l'échelle entre 8 et 9.5 pour mieux voir les écarts
plt.title('Top 10 des Genres les mieux notés')
plt.ylabel('Note Moyenne IMDB')
plt.show()
```



## 0.5 Analyse 3 : Combien de films de ce Top 1000 ont été produits chaque année ?

```
[17]: # On groupe par année et on compte
films_par_annee = df_clean.groupby("Released_Year").count().
        orderBy("Released_Year") # Important de trier par année chrono

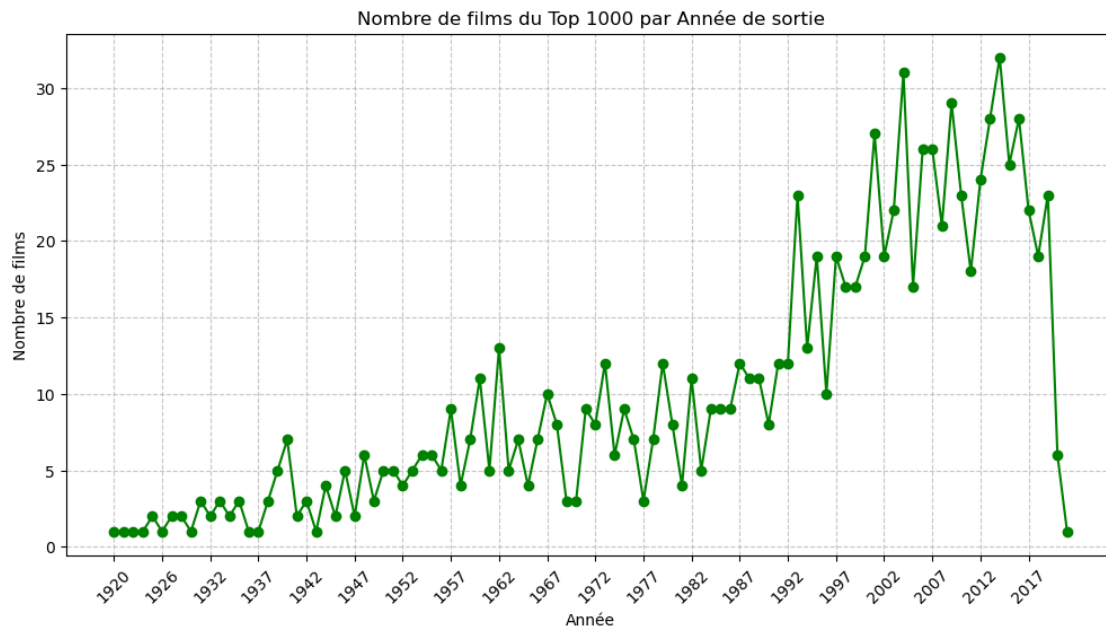
# Conversion en Pandas pour le graphique
pdf_time = films_par_annee.toPandas()
```

### Visualisation 3

```
[18]: # Création du graphique (Courbe)
plt.figure(figsize=(12, 6))
plt.plot(pdf_time['Released_Year'], pdf_time['count'], color='green',
        ↪marker='o', linestyle='-')

# Pour la lisibilité
plt.xticks(pdf_time['Released_Year'][:,5], rotation=45)

plt.title('Nombre de films du Top 1000 par Année de sortie')
plt.xlabel('Année')
plt.ylabel('Nombre de films')
plt.grid(True, linestyle='--', alpha=0.7) # Ajoute une grille pour la lisibilité
plt.show()
```



### 0.6 Analyse 4 : Quels sont les films qui ont rapporté le plus d'argent au Box Office ?

```
[19]: top_gross = df_clean.select("Series_Title", "Gross").orderBy(desc("Gross")).
        ↪limit(10)

top_gross.show() # Affiche les chiffres bruts

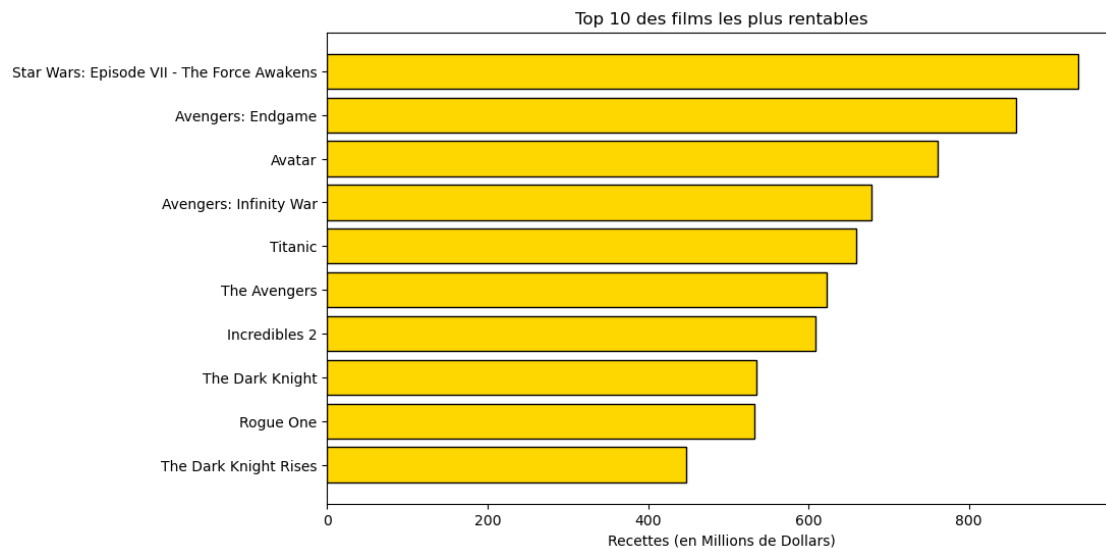
# Visualisation
pdf_gross = top_gross.toPandas()
```

| Series_Title         | Gross        |
|----------------------|--------------|
| Star Wars: Episod... | 9.36662225E8 |
| Avengers: Endgame    | 8.58373E8    |
| Avatar               | 7.60507625E8 |
| Avengers: Infini...  | 6.78815482E8 |
| Titanic              | 6.59325379E8 |
| The Avengers         | 6.23279547E8 |
| Incredibles 2        | 6.08581744E8 |
| The Dark Knight      | 5.34858444E8 |
| Rogue One            | 5.32177324E8 |
| The Dark Knight R... | 4.48139099E8 |

#### Visualisation 4

```
[20]: plt.figure(figsize=(10, 6))
# On divise par 1 million pour que ce soit lisible sur l'axe
plt.barh(pdf_gross['Series_Title'], pdf_gross['Gross'] / 1000000, color='gold',
         edgecolor='black')

plt.xlabel('Recettes (en Millions de Dollars)')
plt.title('Top 10 des films les plus rentables')
plt.gca().invert_yaxis() # Le premier en haut
plt.show()
```



```
[21]: import time
```

## 0.7 PARTIE 3 (BONUS) : CALCUL D'IMPACT ÉCOLOGIQUE

Pour les constantes, nous sommes parti des caractéristiques de l'ordinateur de Camille (Victus 15FB) :

Batterie Li-ion polymère 3 cellules 52,5 Wh

Type d'alimentation électrique avec adaptateur secteur 120 W

Sources référence carbone : EcoInfo CNRS, GreenIT.fr

```
[22]: PUISSANCE_CHARGEUR_MAX = 120.0 # Mon adaptateur 120 W
BATTERIE_CAPACITE_WH = 52.5      # Ma batterie 52.5 Wh
FACTEUR_CHARGE_CPU = 0.65        # Estimation : Docker + Spark utilisent ~65% de
    ↪ la puissance max
MIX_CO2_FRANCE_G_KWH = 57.0      # En France (Source: EcoInfo / RTE)

# Estimation du temps passé
DUREE_SESSION_HEURES = 16.0

def calculer_impact_reel(puissance_max, facteur, heures, mix_co2):
    # Puissance moyenne utilisée = Max * Facteur d'utilisation
    puissance_moyenne = puissance_max * facteur

    # Consommation en kWh
    conso_kwh = (puissance_moyenne * heures) / 1000

    # Impact CO2
    impact_g_co2 = conso_kwh * mix_co2

    return puissance_moyenne, conso_kwh, impact_g_co2

# 2. Calcul
p_moy, kwh, co2 = calculer_impact_reel(PUISSANCE_CHARGEUR_MAX,
    ↪ FACTEUR_CHARGE_CPU, DUREE_SESSION_HEURES, MIX_CO2_FRANCE_G_KWH)

# 3. Affichage professionnel pour le PDF
print(f"--- RAPPORT D'IMPACT ÉCOLOGIQUE (Données Matérielles) ---")
print(f"Matériel : PC Portable avec alimentation
    ↪ {int(PUISSANCE_CHARGEUR_MAX)}W")
print(f"Batterie : Li-ion polymère {BATTERIE_CAPACITE_WH} Wh")
print(f"Charge de travail : Cluster Hadoop/Spark (Estimation charge :
    ↪ {int(FACTEUR_CHARGE_CPU*100)}%)")
print(f"Durée du traitement : {DUREE_SESSION_HEURES} Heures")
print(f"-"*50)
print(f"Puissance moyenne estimée : {p_moy:.1f} Watts")
print(f"Consommation Électrique : {kwh:.4f} kWh")
print(f"Empreinte Carbone : {co2:.2f} gCO2e")
print(f"-"*50)
```

```
print("Méthodologie : Calcul basé sur la puissance de l'adaptateur secteur_␣  
↳ pondérée")  
print("par l'usage CPU/RAM intensif des conteneurs Docker.")
```

--- RAPPORT D'IMPACT ÉCOLOGIQUE (Données Matérielles) ---

Matériel : PC Portable avec alimentation 120W

Batterie : Li-ion polymère 52.5 Wh

Charge de travail : Cluster Hadoop/Spark (Estimation charge : 65%)

Durée du traitement : 16.0 Heures

-----  
Puissance moyenne estimée : 78.0 Watts

Consommation Électrique : 1.2480 kWh

Empreinte Carbone : 71.14 gCO2e  
-----

Méthodologie : Calcul basé sur la puissance de l'adaptateur secteur pondérée  
par l'usage CPU/RAM intensif des conteneurs Docker.